```
================================================================================
LOT-01 — LOT API CONNECTOR (device ↔ lot-systems.com)
================================================================================
DOCUMENT    HW-06 / API
ISSUE       2026.11.12 · REV A
PROTOCOL    v1 · HTTPS · JSON · Bearer device token
CODE        hardware/software/server/device-core.mjs   (logic, zero deps)
            hardware/software/server/http-adapter.mjs  (simulator transport)
            hardware/software/server/fastify-plugin.ts (production wiring, reference)
TEST        node hardware/software/simulator/lot01-sim.mjs  → 11/11 PASS
================================================================================
```

1 · FLOW

```
User on lot-systems.com            LOT server                    LOT-01
─────────────────────              ──────────                    ──────
Settings → Devices → Add ──►  POST /api/devices/pairing-code
                              ◄── code 482913 (10 min)
types code into phone ──────────  BLE provisioning ──────────►  Wi-Fi + code
                                    POST /api/device/pair ◄──  {code, hw_id, fw}
                                    ──► {device_id, token}  ►  stored in NVS

Memory Engine / scheduler ──► notify(user, "Coffee time! ☕")
                              compress → "Coffee time."
                                    GET /api/device/inbox ◄──  every next_poll_s
                                    ──► messages[] ──────────► screen + buzz
                                    POST /api/device/ack  ◄──
press COPY ─────────────────────  POST /api/device/copy ◄──  {message_id, readings}
Log tab shows new entry ◄────  Log {event: device_copy, text: "Coffee time."}
```

2 · ENDPOINTS

All under `/api/device`. All except `/pair` need `Authorization: Bearer <token>`.

Method + path	Body / query	Reply
`POST /pair`	`{code:"482913", hw_id:"LOT01-…", fw:"0.1.0"}`	`201 {device_id, token, protocol:1}` · `403` bad/used code
`GET /inbox`	`?since=<cursor>&h=<local hour>`	`200 {messages:[{id,text,kind,buzz,ts}], cursor, time, next_poll_s}`
`POST /ack`	`{ids:[…]}` (max 20)	`200 {ok:true}`
`POST /copy`	`{message_id?, text?, frame_id?, readings?, ts?}`	`201 {log_id}` · `400` if empty
`POST /telemetry`	`{t_c, rh, p_hpa, iaq, gas_class, battery_pct, rssi}`	`202 {ok:true}` · out-of-range fields dropped
`POST /frame`	raw `image/jpeg`, ≤ 64 KB	`201 {frame_id}` · `413` / `415` / `400`

Website side (cookie session): `POST /api/devices/pairing-code` → `{code, expiresInS}`.

3 · COMPRESSION (requirement 8)

Every message is compressed **on the server** before it is queued, so the device never has to think about length:

1. Strip emoji and joiners. `!` at the end becomes `.`.
2. Collapse whitespace to one line.
3. ≤ 64 characters; cut at a word boundary and end with `…`.
4. Otherwise end with a period.

`"Coffee time! ☕"` → `"Coffee time."`. For AI-written messages, the generator prompt also asks for ≤ 8 words; the compressor is the hard limit.

4 · THE COPY → LOG CONTRACT (requirement 16)

A COPY creates one row in the existing `logs` table:

```
        {
          "userId": "<owner of the device>",
          "event": "device_copy",
          "text": "Coffee time.",
          "metadata": {
            "source": "LOT-01", "deviceId": "…", "messageId": "…",
            "frameId": null, "readings": {"t_c": 21.4, "rh": 40, "gas_class": "coffee"},
            "deviceTs": "2026-11-12T09:41:00Z"
          }
        }
```

To show it in the Log tab: add `'device_copy'` to `LogEvent` in `src/shared/types` and render it like a note, with a small "LOT-01" label.

5 · DATA MODEL (migration for P2)

Table	Columns
`devices`	id, user_id, hw_id, fw, token_hash (SHA-256), created_at, last_seen_at, revoked_at
`device_pairing_codes`	code, user_id, expires_at (single use)
`device_messages`	id, user_id, text (≤ 64), kind, buzz, created_at
`device_message_acks`	device_id, message_id, acked_at
`device_telemetry`	device_id, user_id, at, t_c, rh, p_hpa, iaq, gas_class, battery_pct, rssi
`device_frames`	id, device_id, user_id, at, bytes (or object-storage key)

Tokens are stored only as hashes. Revoking a device sets `revoked_at`.

6 · WHERE NOTIFICATIONS COME FROM

Hook points that already exist in the server:

- `src/server/scheduled-jobs.ts` — add a job that picks routine moments (*"Coffee time."*, *"Evening check-in."*) per user and calls `notify()`.
- Memory Engine / interventions (`/api/interventions`) — a compassionate intervention can also go to the device.
- Weather (`/api/weather`) — rain or temperature change → short message.
- Admin live message (`/api/live-message`) — broadcast to all devices.

7 · LIMITS AND SAFETY

- Rate limits: pair 10/min per IP, copy 30/min, frame 10/min per device.
- Frames: JPEG only, magic-byte check, 64 KB cap.
- Readings: whitelisted numeric fields with physical ranges.
- A device sees only its owner's messages and writes only its owner's Log.